

1) File Handling

Question: Create a file named test.txt and write "Hello Linux" into it.

API: open(), write(), close()

```
#include<stdio.h>
#include<fcntl.h>
#include<unistd.h>

int main()
{
    int fd;
    char msg[]="Hello Linux";

    fd=open("test.txt", O_CREAT|O_WRONLY, 0644);

    write(fd, msg, sizeof(msg)-1);

    close(fd);

    printf("File created and data written\n");
    return 0;
}
```

2) Process

Question: Display the Process ID (PID) of the running program.

API: getpid()

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    printf("Process ID: %d\n", getpid());

    return 0;
}
```

3) Child Process

Question: Create a child process and print message from both parent and child.

API: fork()

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("Hello from Child Process\n");
    }
    else
    {
        printf("Hello from Parent Process\n");
    }

    return 0;
}
```

4) Threads

Question: Create a thread that prints "Hello Thread".

API: pthread_create(), pthread_join()

```
#include<stdio.h>
#include<pthread.h>

void *fun(void *arg)
{
    printf("Hello Thread\n");
    return NULL;
}

int main()
{
```

```
pthread_t t;  
  
pthread_create(&t,NULL,fun,NULL);  
  
pthread_join(t,NULL);  
  
return 0;  
}
```

5) Mutex

Question: Create two threads and use mutex before printing a message.

API: pthread_mutex_lock(), pthread_mutex_unlock()

```
#include<stdio.h>  
#include<pthread.h>  
  
pthread_mutex_t lock;  
void *fun(void *arg)  
{  
    pthread_mutex_lock(&lock);  
    printf("Thread executing safely\n");  
    pthread_mutex_unlock(&lock);  
    return NULL;  
}  
  
int main()  
{  
    pthread_t t1,t2;  
  
    pthread_mutex_init(&lock,NULL);  
  
    pthread_create(&t1,NULL,fun,NULL);  
    pthread_create(&t2,NULL,fun,NULL);  
  
    pthread_join(t1,NULL);  
    pthread_join(t2,NULL);  
  
    pthread_mutex_destroy(&lock);  
  
    return 0;  
}
```

6) Semaphore

Question: Create two threads and use a semaphore to allow only one thread to print at a time.

API: sem_init(), sem_wait(), sem_post()

```
#include<stdio.h>
#include<pthread.h>
#include<semaphore.h>

sem_t sem;

void *fun(void *arg)
{
    sem_wait(&sem);

    printf("Thread executing safely\n");

    sem_post(&sem);

    return NULL;
}

int main()
{
    pthread_t t1,t2;

    sem_init(&sem,0,1);

    pthread_create(&t1,NULL,fun,NULL);
    pthread_create(&t2,NULL,fun,NULL);

    pthread_join(t1,NULL);
    pthread_join(t2,NULL);

    sem_destroy(&sem);

    return 0;
}
```

7) Unnamed Pipe

Question: Send message "Hello Child" from parent to child process through a pipe.

API: pipe(), read(), write()

```
#include<stdio.h>
#include<unistd.h>
#include<string.h>

int main()
{
    int fd[2];
    char msg[]="Hello Child";
    char buffer[20];

    pipe(fd);

    if(fork()==0)
    {
        read(fd[0],buffer,sizeof(buffer));
        printf("Child received: %s\n",buffer);
    }
    else
    {
        write(fd[1],msg,strlen(msg)+1);
    }

    return 0;
}
```

8) Named Pipe (FIFO)

Question: Write a message into named pipe and read it from another program.

API: mkfifo(), open(), read(), write()

Writer:

```
#include<stdio.h>
#include<fcntl.h>
#include<unistd.h>

int main()
{
    int fd;

    mkfifo("myfifo",0666);

    fd=open("myfifo",O_WRONLY);
    write(fd,"Hello FIFO",11);
    close(fd);
    return 0;
}
```

Reader:

```
#include<stdio.h>
#include<fcntl.h>
#include<unistd.h>

int main()
{
    int fd;
    char buf[20];

    fd=open("myfifo",O_RDONLY);
    read(fd,buf,sizeof(buf));

    printf("%s\n",buf);
    close(fd);
    return 0;
}
```

9) Shared Memory

Question: Store "Hello Shared Memory" in shared memory and display it.

API: shmget(), shmat()

```
#include<stdio.h>
#include<sys/shm.h>
#include<string.h>

int main()
{
    int id;
    char *ptr;

    id=shmget(1234,1024,IPC_CREAT|0666);

    ptr=shmat(id,NULL,0);

    strcpy(ptr,"Hello Shared Memory");

    printf("%s\n",ptr);

    shmdt(ptr);

    return 0;
}
```

10) Message Queue

Question: Send "Hello Queue" using message queue and receive it.

API: msgget(), msgsnd(), msgrcv()

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main()
{
    int server, client;
    char buffer[100];

    struct sockaddr_in addr;

    server = socket(AF_INET, SOCK_STREAM, 0);

    addr.sin_family = AF_INET;
    addr.sin_port = htons(8080);
    addr.sin_addr.s_addr = INADDR_ANY;

    bind(server, (struct sockaddr*)&addr, sizeof(addr));

    listen(server, 5);

    printf("Waiting for client...\n");

    client = accept(server, NULL, NULL);

    recv(client, buffer, sizeof(buffer), 0);

    printf("Message received: %s\n", buffer);

    close(client);
    close(server);

    return 0;
}
```


12. UDP Socket

Question:

Send a message from a UDP sender and display it at receiver.

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main()
{
    int sock;
    char buffer[100];

    struct sockaddr_in addr;

    sock = socket(AF_INET, SOCK_DGRAM, 0);

    addr.sin_family = AF_INET;
    addr.sin_port = htons(8080);
    addr.sin_addr.s_addr = INADDR_ANY;

    bind(sock, (struct sockaddr*)&addr, sizeof(addr));

    recvfrom(sock, buffer, sizeof(buffer), 0, NULL, NULL);

    printf("Received: %s\n", buffer);

    close(sock);

    return 0;
}
```